

V3.6 Design Doc

Crystallization as Phase Transition in Neural Agent Dynamics

Status: Draft | Target: Implementazione & Verifica Formale

Autore: MrPie

Data: 9 Febbraio 2026 (Europe/Rome)

Parole chiave: agente neuro-simbolico, reservoir computing, echo state networks, stabilità, Lyapunov, append-only log, SMT, Z3, auditabilità.

Abstract

Si formalizza un agente ibrido neuro-simbolico in cui il processo decisionale è disaccoppiato dalla generazione linguistica. Il sottosistema decisionale (StateEngine) è un sistema dinamico deterministico a tempo discreto basato su Reservoir Computing (ESN/LSM), con pesi ricorrenti fissi e readout lineari statici in inferenza. Il sottosistema di articolazione (LLM) è modellato come attuatore non lineare privo di autonomia decisionale: produce testo o azioni strutturate esclusivamente entro vincoli numerici e logici emessi dal Controller e applicati da un esecutore non bypassabile. La memoria a lungo termine è modellata come Cristallizzazione: una transizione di fase che converte dinamiche fluide (variabili lente, evidenze) in vincoli discreti immutabili append-only. Tale scelta rende l'evoluzione comportamentale auditabile e consente metodi di verifica formale (aritmetica degli intervalli e SMT) sui parametri di controllo e sul gating degli strumenti. Si forniscono definizioni, semantica operativa, condizioni di stabilità (ESP, Lyapunov), specifica del registro dei vincoli e obbligazioni di prova per la safety.

Indice

Abstract	2
Indice	3
1. Introduzione	5
2. Notazione e assunzioni	5
2.1 Spazi e variabili	5
2.2 Assunzioni operative (A1-A10)	5
3. Formalizzazione dell'agente e semantica operativa	6
3.1 Transizione di stato per tick	6
4. StateEngine: dinamica Reservoir Computing	6
4.1 Limitatezza degli stati	6
5. Readout lineare e kernel di controllo	6
5.1 Gating degli strumenti	6
6. Stabilità: Echo State Property e contrattività	7
6.1 Condizione sul raggio spettrale	7
7. Kill-switch via Lyapunov	7
7.1 Condizione sufficiente (linearizzazione)	7
8. Cristallizzazione come transizione di fase	7
8.1 Variabile lenta α_t	7
8.2 Evento di cristallizzazione	7
8.3 Enforcement: proiezione Π_K e maschera M_K	8
8.4 Invarianza dei controlli	8
9. Auditabilità: log append-only e stratigrafia	8
10. Plasticità controllata (opzionale)	8
11. Verifica formale: SMT e intervalli	8

11.1 Obbligazioni di prova (P1-P6) 9

11.2 Bounding di tanh e Norm 9

11.3 Riduzione a vincoli lineari per P2 9

12. Specifica dei contratti e delle strutture dati 9

12.1 Contratto Controller→Esecutore→LLM 9

12.2 Formato di k e risoluzione conflitti 9

13. Limiti e perimetro di validità 9

14. Conclusioni 9

Riferimenti essenziali 10

Appendice A - Checklist di determinismo 11

Appendice B - Scelta di P (schema) 11

Appendice C - Approssimazione piecewise per tanh 11

Appendice D - Schema minimo del registro K 11

Appendice E - Obbligazioni di prova riassunte 11

1. Introduzione

L'obiettivo è costruire un agente conversazionale con strumenti in cui le proprietà di sicurezza e auditabilità dipendano da un nucleo decisionale verificabile e non da un modello linguistico opaco. Il principio architetturale è la separazione fra (i) mantenimento dello stato e decisione (Controller) e (ii) realizzazione linguistica (Actuator).

Contributi. (i) formalizzazione dell'agente come composizione di sistema dinamico deterministico + attuatore vincolato; (ii) adozione di reservoir ricorrente dissipativo con stati limitati; (iii) memoria a lungo termine come transizione di fase che genera vincoli discreti immutabili; (iv) specifica di obbligazioni di prova e di un perimetro verificabile che confina l'incertezza residua al solo livello sintattico del linguaggio.

2. Notazione e assunzioni

Tempo discreto: $t \in \mathbb{N}$. Un tick è un passo sincrono del sistema. Gli spazi vettoriali sono reali e finito-dimensionali.

2.1 Spazi e variabili

Definizione 2.1 (Input aggregato). L'input $u_t \in U \subset \mathbb{R}^M$ è definito come fusione deterministica di tre componenti:

$$u_t = [u_{\text{user}} \oplus u_{\text{context}} \oplus u_{\text{tool_obs}}]$$

Definizione 2.2 (Stato e controlli). Il reservoir è $r_t \in R \subset \mathbb{R}^N$. Lo stato esplicito è $x_t \in X \subset \mathbb{R}^S$. Il controllo è $c_t \in C \subset \mathbb{R}^D$. L'output finale è $y_t \in Y$, dove Y è l'unione disgiunta di testo e azioni JSON.

Convenzione: $\oplus$ denota concatenazione o fusione deterministica di embedding. Le mappe LLM($\cdot$) sono considerate esterne al kernel verificato.

2.2 Assunzioni operative (A1-A10)

A1 (Input limitato). Esiste U_{max} tale che $\|u_t\|_{\infty} \leq U_{\text{max}}$ per ogni t .

A2 (Determinismo del Controller). A parità di pesi, seed e input, F_K e i readout producono lo stesso (r_{t+1}, x_t, c_t) .

A3 (Esecutore non bypassabile). Qualsiasi azione verso tool passa per un esecutore E che applica vincoli hard derivati da (c_t, K) .

A4 (Non-linearità limitata). $\phi = \tanh$ è applicata componente per componente e mappa $\mathbb{R} \rightarrow (-1, 1)$.

A5 (Readout statico in inferenza). $V_{\text{state}}, V_{\text{control}}$ sono fissate durante inferenza.

A6 (Append-only). K cresce solo per append di eventi firmati e hashati.

A7 (Stop). Esiste u_{stop} tale che, se applicato per T tick, porta r_t in una regione di quiete Ω_0 .

A8 (LLM come attuatore). L'LLM non modifica direttamente r_t né K ; emette solo una proposta y_{raw} .

A9 (Accettazione). L'output effettivo y_t è $y_t = \text{Accept}(y_{\text{raw}}; c_t, K)$, con Accept deterministico e conservativo.

A10 (Decidibilità dei predicati). I predicati in K appartengono a una DSL decidibile e valutabile in tempo finito.

3. Formalizzazione dell'agente e semantica operativa

Definizione 3.1 (Agente). Un agente è una tupla:

$$\square = (SE, LLM, E, K)$$

dove SE è lo StateEngine, LLM è l'attuatore, E è l'esecutore/ambiente, K è il registro dei vincoli.

3.1 Transizione di stato per tick

Definizione 3.2 (Semantica di tick). Dato (r_t, K_t) e gli osservabili al tick t , la transizione è:

$$\begin{aligned} u_t &= \text{Encode}(\text{user}_t, \text{context}_t, \text{tool_obs}_t) \\ r_{t+1} &= F_K(r_t, u_t) \\ x_t &= G_{\text{state}}(r_t) \\ c_t &= G_{\text{control}}(r_t) \\ y_{\text{raw}} &= LLM(\text{Prompt}(u_t, h_t), \text{Constraints}(c_t, K_t)) \\ y_t &= \text{Accept}(y_{\text{raw}}; c_t, K_t) \\ K_{t+1} &= K_t \text{ oppure } K_t \cup \{k_{\text{new}}\} \end{aligned}$$

Definizione 3.3 (Accettazione). Accept è un filtro hard che impone: (i) validità di schema; (ii) gating strumenti; (iii) applicazione di override/clamp; (iv) rifiuto in caso di violazione. Per A3, l'LLM non può aggirare Accept.

4. StateEngine: dinamica Reservoir Computing

Si usa un reservoir ricorrente con pesi ricorrenti fissi e proiezione d'ingresso fissa. Per controllare la memoria si adotta la variante leaky-ESN ($\tau \in (0,1]$):

$$\begin{aligned} z_{t+1} &= W_{\text{res}} r_t + W_{\text{in}} u_t + b \\ r_{t+1} &= (1-\tau) r_t + \tau \cdot \tanh(z_{t+1}) \end{aligned}$$

4.1 Limitatezza degli stati

Lemma 4.1. Sotto A4, vale $\|r_t\|_{\infty} < 1$ per ogni $t \geq 1$.

Dimostrazione. $\tanh(z) \in (-1,1)$ componente per componente. r_{t+1} è combinazione convessa di r_t e $\tanh(z_{t+1})$. Per induzione segue che ogni componente resta in $(-1,1)$.

5. Readout lineare e kernel di controllo

Definizione 5.1 (Readout). Si definiscono mappe lineari statiche:

$$\begin{aligned} x_t &= V_{\text{state}} r_t \\ \hat{c}_t &= V_{\text{control}} r_t \\ c_t &= \text{Norm}(\hat{c}_t) \end{aligned}$$

Norm($\cdot$) è una normalizzazione deterministica verso intervalli prefissati. Definizione 5.2 (Insieme sicuro). Si fissa $C_{\text{safe}} = \Pi_d [c_{\text{min}}(d), c_{\text{max}}(d)] \subset C$.

5.1 Gating degli strumenti

Definizione 5.3 (Tool gating). Sia $g_{\text{tool}} = c_t[\text{idx}_{\text{tool}}] \in [0,1]$ un gate. Una richiesta di tool è ammissibile solo se $g_{\text{tool}} \geq \theta_{\text{tool}}$ e nessuna regola in K la vieta. Formalmente, per ogni azione a_t : Allowed($a_t; c_t, K$) è un predicato decidibile applicato da E.

6. Stabilità: Echo State Property e contrattività

Si richiede dissipatività: in assenza di input informativo, lo stato deve contrarre verso una regione limitata; con input limitati, la traiettoria deve restare in un compatto.

6.1 Condizione sul raggio spettrale

Assunzione di progetto: W_{res} è scalata affinché $\rho(W_{\text{res}}) < 1$, dove ρ è il raggio spettrale.

Proposizione 6.1 (Contrattività locale, bozza). Se $\tanh$ è 1-Lipschitz e $\|W_{\text{res}}\|_2 < 1$, allora la mappa $r \mapsto \tanh(W_{\text{res}} r + a)$ è contrattiva in norma 2; segue una forma di Echo State Property per ingressi limitati.

Bozza di dimostrazione. Per due stati r, r' vale $\|\tanh(W_{\text{res}} r + a) - \tanh(W_{\text{res}} r' + a)\|_2 \leq \|W_{\text{res}}(r - r')\|_2 \leq \|W_{\text{res}}\|_2 \|r - r'\|_2$. Se $\|W_{\text{res}}\|_2 < 1$, la costante di Lipschitz è < 1 .

7. Kill-switch via Lyapunov

Definizione 7.1 (Energia). Si definisce una funzione quadratica:

$$V(r_t) = r_t^T P r_t, \text{ con } P \text{ simmetrica e definita positiva}$$

Per definita positiva si intende: $P = P^T$ e $x^T P x > 0$ per ogni $x \neq 0$.

Obiettivo: sotto u_{stop} , imporre $\Delta V = V(r_{t+1}) - V(r_t) < 0$ fino all'ingresso in Ω_0 .

7.1 Condizione sufficiente (linearizzazione)

Per analisi formale si considera una linearizzazione/bounding della dinamica in un insieme compatto Ω . Sia A una matrice che sovrastima la jacobiana della dinamica in Ω . Una condizione sufficiente di stabilità quadratica è l'esistenza di P definita positiva e Q definita positiva tali che:

$$A^T P A - P = -Q$$

L'equazione sopra implica $V(r_{t+1}) - V(r_t) = -r_t^T Q r_t \leq 0$ nel caso lineare, e fornisce un template verificabile per il caso bound. Una procedura standard è risolvere un problema semidefinito offline (Appendice B).

8. Cristallizzazione come transizione di fase

L'adattamento a lungo termine avviene per eventi discreti che aggiungono vincoli al registro K . La transizione è attivata da una variabile lenta che accumula evidenza.

8.1 Variabile lenta α_t

$$\alpha_{t+1} = \alpha_t + g(x_t, u_t, \text{feedback}_t)$$

g è deterministica e limitata. α_t evolve su scala temporale più lenta rispetto a r_t .

8.2 Evento di cristallizzazione

$$\text{se } \alpha_t \geq \alpha^* \text{ allora } K_{t+1} = K_t \cup \{k_{\text{new}}\}$$

Definizione 8.1 (Vincolo). Un vincolo è $k = (\text{pred}, \text{act}, \text{priority}, \text{scope})$, con pred decidibile e act un operatore di enforcement su (c_t, r_{t+1}) o sulle azioni verso tool.

8.3 Enforcement: proiezione Π_K e maschera M_K

Definizione 8.2 (Applicazione dei vincoli). La valutazione di K è deterministica: i vincoli sono ordinati per priority e applicati in sequenza, producendo un operatore complessivo Π_K su C e, opzionalmente, un operatore M_K su R .

Formulazione (i) Enforced-control: $c_t \leftarrow \Pi_K(c_t, meta_t)$, dove Π_K implementa clamp/proibizioni/override.

Formulazione (ii) Enforced-dynamics: $r_{t+1} \leftarrow M_K(r_t, u_t, meta_t) \odot F(r_t, u_t)$, con $M_K \in [0,1]^N$.

Nel caso (ii) la maschera M_K può essere diagonale (selezione) o a tratti (attenuazione), e serve a rimuovere bacini di attrazione associati a comportamenti non ammessi.

8.4 Invarianza dei controlli

Teorema 8.1 (Invarianza). Sia $A_K(meta_t) \subset C$ l'insieme dei controlli ammessi dai vincoli attivi. Se Π_K è tale che $\Pi_K(c, meta) \in A_K(meta)$ per ogni c , allora applicando Π_K a ogni tick si ottiene $c_t \in A_K(meta_t)$ per ogni t .

Dimostrazione. Diretta per chiusura dell'immagine di Π_K . QED.

9. Auditabilità: log append-only e stratigrafia

Definizione 9.1 (Evento). Un evento di append è un record:

$$e_t = (t, \alpha_t, k_{new}, justification, signature, prev_hash, hash)$$

con $hash = H(\text{serialize}(e_t \text{ senza hash}))$. La catena $prev_hash$ rende rilevabile qualsiasi modifica retroattiva. La 'geologia' del comportamento è la sequenza temporale degli e_t che spiegano cambiamenti qualitativi in Π_K e M_K .

10. Plasticità controllata (opzionale)

Per memoria a breve termine si può attivare una plasticità locale deterministica in modalità Learning; in inferenza è disattivata.

$$w_{ij}(t+1) = w_{ij}(t) + \eta \cdot r_i(t) \cdot r_j(t)$$

Ogni sessione Learning produce uno snapshot o diff hashato; la riproducibilità è garantita selezionando esplicitamente lo snapshot.

11. Verifica formale: SMT e intervalli

Il perimetro verificabile include: bounds su c_t , correttezza del gating tool, append-only di K , e proprietà di stop. L'LLM è esterno al kernel verificato e viene mediato da Accept.

11.1 Obbligazioni di prova (P1-P6)

P1 (Bounds). $\forall t: c_t \in C_{\text{safe}}$.

P2 (Tool gating). $\forall t: \text{tool_request} = \text{Allowed}(\text{tool_action}; c_t, K)$.

P3 (Schema). $\forall t: y_t$ è testo o JSON valido; altrimenti Reject.

P4 (Stop). Se u_{stop} è applicato per T tick, allora $r_t \in \Omega_0$ e c_t si porta in una configurazione di quiete.

P5 (Append-only). K cresce solo per append; non esistono update/del.

P6 (Determinismo). A parità di input e snapshot, la traiettoria (r_t, c_t, K_t) è riproducibile.

11.2 Bounding di tanh e Norm

Per SMT, tanh e σ possono essere approssimate con funzioni a tratti lineari con errore ε . Alternativamente, poiché Lemma 4.1 fornisce $r_t \in (-1,1)^N$, è possibile usare bound intervallari conservativi per $\hat{c}_t = V_{\text{control}} r_t$ e affidare il rispetto di P1 al clamp Π_K e/o a un clamp finale verso C_{safe} .

11.3 Riduzione a vincoli lineari per P2

$\text{Allowed}(\text{tool_action}; c_t, K)$ si esprime come formula booleana su predicati decidibili della DSL e confronti su componenti di c_t . Dato che K è finito, si ottiene una formula con disgiunzioni finite; la parte numerica è lineare o bounded.

12. Specifica dei contratti e delle strutture dati

Questa sezione fissa requisiti di interfaccia. La correttezza richiede che i flussi di dati rispettino tali contratti.

12.1 Contratto Controller→Esecutore→LLM

Il Controller emette (x_t, c_t) . L'esecutore costruisce il prompt e applica Accept. L'LLM restituisce y_{raw} come proposta. L'esecutore decide y_t e (eventualmente) tool actions, sempre sotto Π_K .

12.2 Formato di k e risoluzione conflitti

I vincoli sono valutati in ordine di priority; a parità di priority si usa l'ordine di inserimento (append order). In caso di conflitto, prevale l'azione più restrittiva (principio conservativo) definita a livello di DSL.

13. Limiti e perimetro di validità

Il modello dimostra proprietà su parametri di controllo e accesso agli strumenti; non dimostra verità del contenuto linguistico. La safety intenzionale è quindi delegata al kernel verificabile (c_t, K, E) .

14. Conclusioni

La separazione decisione/articolazione, combinata con reservoir dissipativo e cristallizzazione append-only, produce un agente auditabile: ogni cambiamento qualitativo è tracciabile e le componenti critiche sono verificabili con tecniche SMT/intervallari. L'incertezza residua è confinata alla superficie linguistica.

Riferimenti essenziali

- H. Jaeger, “The ‘Echo State’ Approach to Analysing and Training Recurrent Neural Networks”, 2001.
- W. Maass, T. Natschläger, H. Markram, “Real-Time Computing Without Stable States...”, 2002.
- S. Boyd, L. Vandenberghe, “Convex Optimization”, 2004. (LMIs).
- L. de Moura, N. Bjørner, “Z3: An Efficient SMT Solver”, 2008.

Appendice A - Checklist di determinismo

- Seed fissato per inizializzazione di W_{res} e W_{in} e per qualsiasi sampling di supporto.
- Aritmetica deterministica: stessa precisione; evitato parallelismo non deterministico nei path critici.
- Encode($\cdot$) deterministico: stessa versione di embedding, stessa normalizzazione e tokenizer; backend stabile quando richiesto.
- Ordine di riduzioni (somma/matmul) fissato o demandato a backend deterministico.
- Snapshot immutabili di (W_{res} , W_{in} , b , V_{state} , $V_{control}$) con hash e versioning.

Appendice B - Scelta di P (schema)

Nel caso linearizzato $r_{t+1} = A r_t + B u_t$, la stabilità quadratica si ottiene trovando P definita positiva tale che $A^T P A - P = -Q$ con Q definita positiva. La ricerca può essere posta come programma semidefinito offline.

Appendice C - Approssimazione piecewise per tanh

$\tanh$ può essere approssimata con segmenti lineari su $[-a, a]$ e saturazione fuori intervallo. L'errore massimo ε è precomputabile e inseribile come vincolo aggiuntivo in SMT.

Appendice D - Schema minimo del registro K

```
{
  "t": <int>,
  "alpha": <float>,
  "k_new": {
    "id": <string>,
    "pred": <dsl>,
    "act": <dsl>,
    "scope": <string>,
    "priority": <int>
  },
  "justification": <string>,
  "prev_hash": <hex>,
  "hash": <hex>,
  "signature": <hex>
}
```

Appendice E - Obbligazioni di prova riassunte

P1-P6 costituiscono il nucleo della verifica. In aggiunta, è necessario dimostrare la non-esistenza di canali laterali che bypassino l'esecutore (A3): qualsiasi invocazione di tool deve attraversare Accept e Π_K .